Name: Danny Mae P. Pabular Section:D
Date:11/11/26

Exercise Title: Campus Cafe Ordering System

1. Explanation of OOP Concepts Used

Conditional Statements: I used if, if-else, and if-else-if statements to check conditions and make choices. These are used to check prices, drink sizes, discounts, and the number of items in an order.

Classes and Objects: I created seven Java classes: MenuItem, Beverage, Pastry, Meal, Customer, Order, and CafeApp. Each class has its own task. I created objects in the main method to run the program.

Attributes and Behaviors: Attributes store data such as names, prices, item codes, sizes, and loyalty points. Methods perform tasks such as adding items, getting prices, and printing receipts.

Object Interaction: The Order class connects the Customer class and MenuItem objects. It handles customer orders, counts items, gets the total price, prints receipts, and adds loyalty points.

Constructors: I used default and parameterized constructors to set the values of objects. The Customer class has three constructors with different parameters.

Encapsulation: I used private and protected variables to keep the data safe. I also used getters and setters to get and change values. I added checks to stop invalid prices and sizes.

Inheritance: I used inheritance by making Beverage, Pastry, and Meal child classes of MenuItem. They can use the common data and methods from the parent class.

Polymorphism: I used method overloading and method overriding. Method overloading is shown in the Customer class through the addPoints() methods. Method overriding is used in Beverage, Pastry, and Meal to calculate prices in different ways.

Abstraction: I created an abstract class called MenuItem with an abstract computePrice() method. Each child class has its own way of getting the item price.

2. Java Naming Conventions

I followed Java naming rules to make my code clean and easy to read. I used PascalCase for class names, such as MenuItem, Customer, and CafeApp. I used camelCase for variables and methods, such as basePrice, itemCode, and computePrice(). I also used clear names to make the code easy to understand. **3. Program Output**

The program shows how the Campus Cafe Ordering System takes orders, gets prices, gives discounts, and adds loyalty points.

First Order – Order #1001

The first customer is Danny Pabular, a student. He ordered a Dark Chocolate (Large) for ₱110, Malunggay Pandesal (Warmed) for ₱75, and Lechon Manok (Upsized with 1 side) for ₱257. The subtotal is ₱442.00. He received a 10% student discount of ₱44.20, making the final total ₱397.80. He earned 7 loyalty points.

The program also tried to add a Cheese Cake, but the order was already full. It also checked a negative price and displayed an error because the base price cannot be negative.

Second Order – Order #1002

The second customer is Anna Hijara. She ordered a Dark Chocolate (Large) for ₱110 and Bistek (Regular, with 0 sides) for ₱285. The subtotal is ₱395.00. She used a ₱20 voucher, making the final total ₱375.00. She earned 7 loyalty points.

Both orders were successfully processed, and the program displayed the "Process Completed" message.

4. Challenges Encountered and Solutions

While making the program, I found it hard to understand how the classes work together. I also had to make sure that inheritance, constructors, and method overriding worked correctly. Another challenge was checking prices and limiting each order to three items. I solved these problems by checking my code, using if-else statements, separating the code into different Java files, and testing the program. This helped me make sure that the prices, discounts, and loyalty points were correct.

5. Conclusion

Through this activity, I learned more about Java Object-Oriented Programming. I learned how to create classes and objects, use constructors, and apply inheritance, polymorphism, encapsulation, and abstraction. I also learned how to connect different classes to make a simple ordering system. This activity helped me improve my coding skills and understand how OOP works in a real program.